

Astrazione dei dati

Parlando di classi abbiamo visto come, tipicamente, la parte relativa ai dati (gli attributi) venga definita privata, in modo da non renderla accessibile e modificabile all'esterno della classe stessa, ragione per la quale vengono messi a disposizione dei metodi per poterlo fare. Tutto questo è racchiuso in un concetto base della programmazione ad oggetti: l'information hiding.

In generale, l'**astrazione dei dati** è quel meccanismo che consente di nascondere alcuni dettagli, rendendo visibile all'utente solamente le informazioni necessarie.

Ci sono due modi per ottenerla:

1. classi astratte;
2. interfacce.

Classi astratte

Una classe astratta è una classe "ristretta", vincolata, che non può essere utilizzata per creare degli oggetti (farlo genererebbe un errore). È, tuttavia, possibile creare delle sottoclassi di una classe astratta.

Per dichiarare una classe come astratta è possibile utilizzare la parola chiave *abstract*.

Metodi astratti

Un metodo astratto è un metodo presente solamente in una classe astratta, per il quale non è fornita una implementazione: è un metodo senza corpo. Se un metodo di una classe è definito come *abstract*, anche la classe dovrà esserlo. I metodi astratti, come detto, non forniscono una implementazione, pertanto dovranno essere implementati nelle sottoclassi.

N.B.: una classe astratta può avere anche metodi classici.

Esempio:

```
abstract class Animale {
    public abstract void verso();
    public void dormi() {
        System.out.println("Zzz");
    }
}

class Mucca extends Animale {
    public void verso() {
        System.out.println("La mucca fa: muuu");
    }
}
```

Nell'esempio, la classe *Animale* è dichiarata come astratta e presenta un metodo astratto ed un metodo tradizionale. La classe *Mucca* è ottenuta per derivazione estendendo la classe *Animale* (è una sua sottoclasse) e deve obbligatoriamente fornire una implementazione del metodo astratto.

Interfacce

Una interfaccia può essere vista come una classe completamente astratta, che comprende una serie di metodi senza implementazione (senza corpo), detti metodi di interfaccia. Per poter utilizzare i metodi di una interfaccia è necessario creare una classe che vada ad implementare tale interfaccia.

Similmente alle classi derivate, che vengono create utilizzando la parola chiave **extends** seguita dal nome della classe originaria, le classi che implementano una interfaccia utilizzeranno la parola chiave **implements**, seguita dal nome dell'interfaccia. All'interno della classe, verrà poi fornita obbligatoriamente una implementazione di tutti i metodi dell'interfaccia.

Esempio:

```
interface Animale {
    public void verso();
    public void dormi();
}

class Mucca implements Animale {
    public void verso() {
        System.out.println("La mucca fa: muuu");
    }
    public void dormi() {
        System.out.println("Zzz");
    }
}
```

Anche le interfacce non possono essere utilizzate per creare degli oggetti: occorre creare una classe che implementi l'interfaccia e, successivamente, istanziare oggetti di tale classe.

N.B.:

1. di default tutti gli attributi di una interfaccia sono definiti come **public**, **static** e **final**;
2. di default tutti i metodi di una interfaccia sono definiti come **public** e **abstract**.

Qual è lo scopo delle interfacce? Abbiamo visto che un limite del linguaggio Java è l'impossibilità, per una classe, di poter estendere più di una classe (ovvero una classe può avere solamente una superclasse).

Questo non è vero per le interfacce: una classe può implementare più di una interfaccia; per farlo, è sufficiente separare i nomi delle interfacce con una virgola. Quindi l'ereditarietà multipla (non supportata in Java) può essere raggiunta sfruttando le interfacce.

Esempio:

```
interface Animale {
    public void verso();
    public void dormi();
}

interface Mammifero{
    public void allatta();
}

class Mucca implements Animale, Mammifero {
    public void verso() {
        System.out.println("La mucca fa: muuu");
    }
    public void dormi() {
        System.out.println("Zzz");
    }
    public void allatta(){
        System.out.println("La mucca allatta il cucciolo");
    }
}
```